SELF-DIRECTED LEARNING PLAN

AI Engineering Roadmap

A seven-phase, project-driven path from API foundations to multi-agent orchestration and capstone integration — built on direct API calls and mechanical understanding rather than framework abstractions.

7	~28	~190	5
PHASES	WEEKS	HOURS	PROJECTS

> *“Skip premature abstractions. Learn the mechanics by building with direct API calls — frameworks come later, once the patterns underneath them are already understood.”*

Roadmap at a Glance

Updated — Phases 4.5 and 5 are now one continuous project: the orchestrator is hand-built first, then converted to MCP.

P1	P2	P3	P4	P4.5	P5	P6
Wks 1–3	Wks 4–7	Wks 8–13	Wks 14–17	Wks 18–20	Wks 21–25	Wks 26–28
Expense Tracker	Calendar Agent	RAG + Evals	MLOps	Orchestration	MCP Capstone	Frameworks

WEEKS 1–3 · ~18 HOURS

1 Smart Expense Tracker DONE

Closing API foundation gaps — auth, request/response handling, error states, persistence.

CORE LEARNING TARGETS

- Structuring a FastAPI app with proper request/response models
- Serving with Gunicorn — process management basics
- Designing clean error states instead of raw stack traces
- First contact with the Claude API for unstructured-to-structured parsing

> **Project:** An expense tracker that takes free-text entries ("$12 lunch with Sam") and parses them into structured records via the Claude API, served through FastAPI + Gunicorn.

RESOURCES & LEARNING MATERIAL

Anthropic Academy — Building with the Claude API OFFICIAL

Skilljar, ~8 hrs. The deepest official course on request structure, error handling, and API patterns.

FastAPI official docs — Tutorial OFFICIAL

fastapi.tiangolo.com/tutorial — work through request bodies, response models, and error handling sections specifically.

Gunicorn docs — Design & Deployment OFFICIAL

docs.gunicorn.org — worker types and process management; short, no need for more than this.

WEEKS 4–7 · ~22 HOURS

2 Natural Language Calendar Agent DONE

Agentic tool loops, built with direct Anthropic API calls — no LangChain.

CORE LEARNING TARGETS

- Mechanical understanding of the tool-use loop: `tool_use` → execute → `tool_result` → continue
- Writing tool descriptions as prompts, not documentation
- Chaining as an emergent model decision, not explicit orchestration code
- Correct placement of human-in-the-loop confirmation (system prompt and tool description)
- Safe conversation-history trimming (avoiding the 400 Bad Request from orphaned `tool_result` blocks)

> **Project:** A Google Calendar agent (`get_events`, `find_free_slots`, create/update/cancel_event, `get_weather`) with a CLI front-end and a Telegram bot stretch goal — confirmed working, including a real encounter with the history-trimming 400 error.

RESOURCES & LEARNING MATERIAL

Anthropic docs — Tool use overview OFFICIAL

docs.claude.com/en/docs/build-with-claude/tool-use — the canonical reference for tool_use/tool_result block structure.

Claude Cookbook — Tool use & function calling OFFICIAL

github.com/anthropics/claude-cookbooks — runnable notebooks showing the full loop, including multi-tool chaining.

Google Calendar API — Python quickstart OFFICIAL

developers.google.com/calendar/api/quickstart/python — covers OAuth consent screen, scopes, and token refresh.

WEEKS 8–13 · ~32 HOURS

3 AI Support Agent: RAG + Eval Harness DONE

Vector databases introduced contextually, exactly when the project needs them.

CORE LEARNING TARGETS

- Chunking strategy and embedding generation — the mechanics, not just calling an API
- ChromaDB locally → similarity search → relevance thresholds
- Building an eval harness: retrieval precision/recall, groundedness checks, hallucination detection
- Where RAG grounding constraints belong (same “close to the decision point” principle from tool descriptions)

> **Project:** A support agent answering grounded questions from a real document corpus, with a working eval harness measuring whether retrieved chunks actually support the generated answer. Built on the FastAPI docs corpus, with a committed k/threshold parameter sweep and separate baseline and final groundedness runs.

RESOURCES & LEARNING MATERIAL

Claude Cookbook — RAG guide OFFICIAL

Naive RAG pipeline plus a dedicated section on evaluating retrieval vs. end-to-end performance separately.

Claude Cookbook — Contextual retrieval guide OFFICIAL

platform.claude.com/cookbook/capabilities-contextual-embeddings-guide — improving chunk relevance with contextual embeddings; read after the basic RAG guide, not before.

ChromaDB docs — Getting Started OFFICIAL

docs.trychroma.com — local persistence, collections, and similarity search basics; you need only the first few pages.

Anthropic Academy — AI Fluency / Evaluations track OFFICIAL

Skilljar — the most relevant addition once retrieval is working.

WEEKS 14–17 · ~24 HOURS

4 MLOps Sprint DONE

Deploy and monitor the Phase 3 RAG agent — taking it from script to service.

CORE LEARNING TARGETS

- Containerizing the RAG agent for deployment
- Basic monitoring: latency, error rates, eval-score drift over time
- Logging tool calls and retrieval results for post-hoc debugging
- What “production-ready” actually requires beyond a working notebook

> **Project:** The Phase 3 RAG agent deployed as a monitored service on ECS Fargate — secrets in Secrets Manager, separate IAM execution and task roles, CloudWatch logs and dashboards, and an eval-drift log tracking retrieval and groundedness as separate fields over time.

RESOURCES & LEARNING MATERIAL

Docker docs — Get Started OFFICIAL

docs.docker.com/get-started — just enough to containerize a FastAPI service correctly.

Anthropic docs — Monitoring & usage OFFICIAL

docs.claude.com — check the platform usage/cost tracking pages for what's available out of the box before building custom logging.

Prometheus + Grafana — Get Started guide COMMUNITY

The standard lightweight stack for self-hosted latency/error dashboards; overkill is easy here, keep it to 2–3 panels.

WEEKS 18–20 · ~16 HOURS

4.5 Multi-Agent Orchestration IN PROGRESS

Sub-agent delegation and context management — via direct API calls, before any framework.

CORE LEARNING TARGETS

- Orchestrator/worker pattern: a “delegate” tool whose implementation is itself a full nested agent loop
- Context isolation vs. sharing: does a sub-agent get full parent history, or a scoped summary?
- Result surfacing: collapsing a sub-agent's internal multi-turn loop into a single `tool_result` block
- Reconciling conflicting sub-agent signals into one recommendation — in the system prompt and eval spec, never hardcoded as branching logic
- Failure propagation: what the orchestrator sees when a sub-agent errors several levels deep
- Chunking and embedding as a function of *document shape* — issue-to-issue matching is symmetric, unlike Phase 3's query-to-document retrieval

> **Project — OSS Maintainer Copilot:** an orchestrator that decides what a maintainer should do with an incoming issue on `pydantic/pydantic`, by delegating to a triage sub-agent (duplicate detection over issue history) and a docs sub-agent (the Phase 3 RAG agent, corpus swapped to pydantic's docs). The two routinely disagree — “likely duplicate of #10590” against “the docs don't cover this case at all” — and reconciling them is the deliverable. Ground truth is harvested from the repo's duplicate-labeled issues; writes go to a personal sandbox repo behind an orchestrator-level approval step.

RESOURCES & LEARNING MATERIAL

Claude Cookbook — Sub-agents notebook OFFICIAL

github.com/anthropics/claude-cookbooks — using a smaller model as a sub-agent under a larger orchestrator; the closest official example to this phase's project.

Claude Code docs — Subagents OFFICIAL

docs.claude.com/en/docs/claude-code/sub-agents — even though this describes Claude Code's own subagents, the context-isolation model it documents is conceptually identical to what you're hand-building.

GitHub REST API — Issues OFFICIAL

docs.github.com/rest/issues — pagination, the `pull_request` field that contaminates issue lists, and the primary vs. secondary rate limits that supply this phase's failure cases.

Anthropic Academy — Subagents course OFFICIAL

Skilljar, ~30–60 min — short, but useful for vocabulary before you build the pattern yourself from scratch.

WEEKS 21–25 · ~30 HOURS

5 MCP Capstone REVISED

The same system as Phase 4.5, with exactly one thing changed: where sub-agent code runs.

CORE LEARNING TARGETS

- Converting hand-written sub-agents into MCP servers without the orchestrator noticing
- Consuming an MCP server you did *not* write — the first tool in the system whose schema you don't control
- Measuring what MCP costs and buys by re-running Phase 4.5's eval unchanged across the transport swap
- Whether a hand-built failure taxonomy survives someone else's error handling
- Exposing your own system as an MCP server others can point at their repo
- End-to-end integration testing across services with different deployment states

> **Project:** Take the Phase 4.5 Maintainer Copilot and (1) convert each sub-agent into an MCP server, with the orchestrator becoming an MCP client; (2) re-run the identical eval to get a real number for MCP's latency cost and composability gain; (3) delete the hand-rolled GitHub REST client in favour of GitHub's official MCP server; (4) add an external MCP you didn't write — Slack, for notifications; (5) publish maintainer-copilot itself as an MCP server anyone can run against their own repository. Step 3 teaches the most, and only works because Phase 4.5 hand-wrote the thing it replaces.

RESOURCES & LEARNING MATERIAL

Model Context Protocol — Official docs OFFICIAL

modelcontextprotocol.io — start with “Core Architecture” then “Build an MCP Server,” in that order.

Anthropic docs — MCP quickstart OFFICIAL

docs.claude.com/en/docs/claude-code/mcp — connects an MCP server end to end; the fastest path to a working first server.

GitHub MCP server OFFICIAL

The drop-in replacement for your Phase 4.5 REST client — read its tool schemas before converting, and note where they don't match the ones you wrote.

Claude Cookbook — MCP examples OFFICIAL

github.com/anthropics/claude-cookbooks — check the `mcp/` directory for server examples close to your use case.

WEEKS 26–28 · ~14 HOURS

6 Frameworks: LangGraph & n8n NEW

Now that the mechanics are second nature, learn the abstractions that wrap them.

CORE LEARNING TARGETS

- Mapping LangGraph's graph/state/node abstractions onto the tool loop you already built by hand
- Recognizing what LangGraph buys you (state persistence, visualization, checkpointing) versus what it hides
- n8n as a visual automation layer — connecting your existing agents into workflows without glue code
- Judging, case by case, when a framework is worth the abstraction cost in real projects

> **Project:** Re-implement the Phase 4.5 orchestrator in LangGraph, then build an n8n workflow that triggers it — comparing both against the hand-built version for clarity, debuggability, and development speed. Optional third arm: the same orchestrator expressed as Claude Code subagents, which is the config-file end of the same spectrum.

RESOURCES & LEARNING MATERIAL

LangGraph official docs — Tutorials OFFICIAL

langchain-ai.github.io/langgraph — start with the “Quickstart” then “Common workflows”; you'll recognize the tool loop immediately.

LangGraph — Multi-agent concepts page OFFICIAL

Read this specifically against your Phase 4.5 build — it's the best way to see what the framework abstracts vs. what you already did by hand.

n8n official docs — Workflow basics OFFICIAL

docs.n8n.io — nodes, triggers, and the HTTP Request node are the only things you need to connect your existing agents.

n8n — AI Agent node documentation OFFICIAL

Covers wiring n8n directly to the Claude API and to MCP servers from Phase 5.

Guiding Principles

What's carried through every phase of this roadmap

Mechanics before abstraction	Every pattern (tool loops, RAG, orchestration) is built once with direct API calls before any framework is introduced. Frameworks are learned last, in Phase 6, once there's a hand-built mental model to compare them against.
Contextual introduction	Tools like vector databases are introduced exactly when a project needs them — not front-loaded. This is why ChromaDB lives in Phase 3, not Phase 1.
Confirmation at the decision point	Constraints (human-in-the-loop confirmation, RAG grounding rules) belong as close to the model's decision point as possible — in tool descriptions, not just system prompts.
One new variable at a time	Phases 4.5 and 5 build one system across two phases, changing only the transport between them. Failure propagation can't be learned while the protocol underneath it is also new.
Improvement is a number	Retrieval, groundedness, and refusal correctness are tracked separately and never blended into a single accuracy score. “I improved it” requires a committed before and after.
Credentials vs. self-direction	Paid courses are skipped in favor of free, high-quality resources (Anthropic's Skilljar curriculum) unless a specific credential has clear employer value.

Skip / Defer / Learn Decisions

LangChain / LangGraph DEFER	Deferred to Phase 6. Abstracts away the tool-loop mechanics that Phases 2–4.5 are designed to teach directly.
n8n DEFER	Deferred to Phase 6. A productivity multiplier for someone who already has working agents to connect — not a foundational tool.
Vector databases LEARN	Learn in Phase 3. Introduced contextually when the RAG project creates a real need for similarity search.
Multi-agent orchestration LEARN	Learn in Phase 4.5. Mechanical, hand-built — sub-agent delegation and context management before any orchestration framework.